

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)
Full Stack Development Using MERN
MINI PROJECT

Roll No.:T029	Name: Vinayak Padhy
Class: TYIT	Batch:01
Date of Assignment: 26-09-26	Date/Time of Submission: 26-09-26

App.jsx(Code):-

```
import { useEffect, useState } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [food, setFood] = useState([]);
  const [orders, setOrders] = useState([]);

  const [name, setName] = useState("");
  const [rollNo, setRollNo] = useState("");
  const [course, setCourse] = useState("");
  const [foodItem, setFoodItem] = useState("");
  const [quantity, setQuantity] = useState(1);

  // Get food items
  const getFood = () =>
  { axios
    .get("http://localhost:3000/food")
    .then((response) =>
      { setFood(response.data);
    })
    .catch((error) =>
      { console.log(error);
    });
  };

  // Get placed orders
  const getOrders = () =>
  { axios
```

```
.get("http://localhost:3000/orders")
.then((response) =>
  { setOrders(response.data);
  })
.catch((error) =>
  { console.log(error);
  });
};

useEffect(() =>
  { getFood();
    getOrders();
  }, []);

// Place order
const placeOrder = () => {
  if (!name || !rollNo || !course || !foodItem || !quantity)
    { alert("Please fill all the details");
      return;
    }

  const selectedFood =
    food.find( (item) => item.name ===
      foodItem
    );

  const total = selectedFood.price * Number(quantity);

  const newOrder = {
    name: name,
    rollNo: Number(rollNo),
    course: course,
    foodItem: foodItem,
    quantity: Number(quantity),
    price: selectedFood.price,
    total: total
  };

  axios
    .post("http://localhost:3000/orders", newOrder)
    .then((response) =>
      { alert(response.data.message);

        setName("");
```

```

    setRollNo("");
    setCourse("");
    setFoodItem("");
    setQuantity(1);

    getOrders();
  })
  .catch((error) =>
    { console.log(error);
      alert("Error placing order");
    });
};

return (
  <div className="container">

    <h1>College Canteen Pre-Order System</h1>

    { /* Food List */}
    <h2>Available Food Items</h2>

    <table border="1">
      <thead>
        <tr>
          <th>Food Item</th>
          <th>Price (₹)</th>
        </tr>
      </thead>

      <tbody>
        {food.map((item) => (
          <tr key={item._id}>
            <td>{item.name}</td>
            <td>₹{item.price}</td>
          </tr>
        ))}
      </tbody>
    </table>

    <hr />

    { /* Order Form */}
    <h2>Student Order Form</h2>

```

```

<input
  type="text"
  placeholder="Student Name"
  value={name}
  onChange={(e) => setName(e.target.value)}
/>

<br />
<br />

<input
  type="number"
  placeholder="Roll Number"
  value={rollNo}
  onChange={(e) => setRollNo(e.target.value)}
/>
<br />
<br />
<input
  type="text"
  placeholder="Course"
  value={course}
  onChange={(e) => setCourse(e.target.value)}
/>

<br />
<br />
<select
  value={foodItem}
  onChange={(e) => setFoodItem(e.target.value)}
>
  <option value="">Select Food Item</option>

  {food.map((item) => (
    <option key={item._id} value={item.name}>
      {item.name} - ₹{item.price}
    </option>
  ))}
</select>

<br />
<br />

```

```

<input
  type="number"
  min="1"
  placeholder="Quantity"
  value={quantity}
  onChange={(e) => setQuantity(e.target.value)}
/>
<br />
<br />

<button onClick={placeOrder}>
  Place Order
</button>
<hr />
{/* Order List */}
<h2>Placed Orders</h2>

<table border="1">
  <thead>
    <tr>
      <th>Name</th>
      <th>Roll No</th>
      <th>Course</th>
      <th>Food Item</th>
      <th>Quantity</th>
      <th>Price</th>
      <th>Total</th>
    </tr>
  </thead>

  <tbody>
    {orders.map((order, index) => (
      <tr key={order._id || index}>
        <td>{order.name}</td>
        <td>{order.rollNo}</td>
        <td>{order.course}</td>
        <td>{order.foodItem}</td>
        <td>{order.quantity}</td>
        <td>₹{order.price}</td>
        <td>₹{order.total}</td>
      </tr>
    ))}
  </tbody>
</table>

```

```

    </tbody>
  </table>

```

```

  </div>
);}
export default App;

```

App.css (Code):-

```

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background-color: #f5f5f5;
}
.container
{ width:
  90%;
  max-width: 1000px;
  margin: 30px auto;
  background-color: white;
  padding: 30px;
  border-radius: 12px;
  box-shadow: 0 4px 15px rgba(0, 0, 0, 0.1);
}
h1 {
  text-align: center;
  margin-bottom: 30px;
}
h2 {
  margin-top: 25px;
  margin-bottom: 15px;
}
table {
  width: 100%;
  border-collapse: collapse;
  margin-bottom: 25px;
}
th,
td {
  border: 1px solid #ddd;
  padding: 10px;
  text-align: center;
}

```

```
th {
  background-color: #eeeeee;
}
input,
select {
  width: 100%;
  max-width: 400px;
  padding: 10px;
  margin-top: 5px;
  border: 1px solid #ccc;
  border-radius: 5px;
  box-sizing: border-box;
}
button {
  padding: 11px 25px;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 16px;
}
button:hover
{ opacity:
  0.85;
}
hr {
  margin: 30px 0;
  border: 0;
  border-top: 1px solid #ddd;
}
```

[Server.js](#) (Code):-

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
const PORT = 3000;

// Middleware
app.use(cors());
app.use(express.json());

// Connect MongoDB
mongoose
```

```
.connect("mongodb://127.0.0.1:27017/CollegeDB")
.then(async () => {
  console.log("MongoDB Connected Successfully");

  // Create food collection/data if it is empty
  const foodCollection = mongoose.connection.db.collection("food");

  const foodCount = await foodCollection.countDocuments();

  if (foodCount === 0) {
    await foodCollection.insertMany([
      {
        name: "Veg Sandwich",
        price: 50
      },
      {
        name: "Masala Dosa",
        price: 60
      },
      {
        name: "Vada Pav",
        price: 30
      },
      {
        name: "Cold Coffee",
        price: 40
      },
      {
        name: "Tea",
        price: 20
      }
    ]);

    console.log("Food items added successfully");
  }
})
.catch((error) => {
  console.log("MongoDB Connection Failed");
  console.log(error);
});

// Home API
app.get("/", (req, res) => {
```

```
res.send("College Canteen Pre-Order System API is Running");
});
// GET /food
// Retrieve available food items
app.get("/food", async (req, res) =>
{ try {
  const food = await mongoose.connection.db
    .collection("food")
    .find()
    .toArray();

  res.json(food);
} catch (error)
{ res.status(500).json({
  message: "Error fetching food items"
});
}
});
// POST /orders
// Add a new food order
app.post("/orders", async (req, res) =>
{ try {
  const order = req.body;

  await mongoose.connection.db
    .collection("orders")
    .insertOne(order);

  res.json({
    message: "Order placed successfully",
    order: order
  });
} catch (error)
{ res.status(500).json({
  message: "Error placing order"
});
}
});
// GET /orders
// Retrieve placed orders
app.get("/orders", async (req, res) =>
{ try {
```

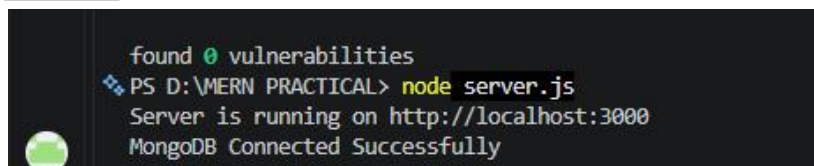
```

const orders = await mongoose.connection.db
  .collection("orders")
  .find()
  .toArray();

res.json(orders);
} catch (error)
{ res.status(500).json({
  message: "Error fetching orders"
});
}
});
// Start server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

```

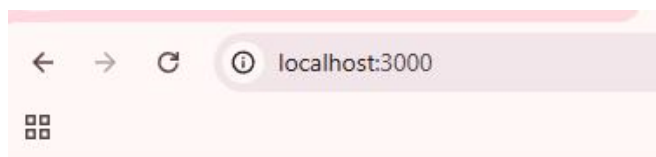
Output:-



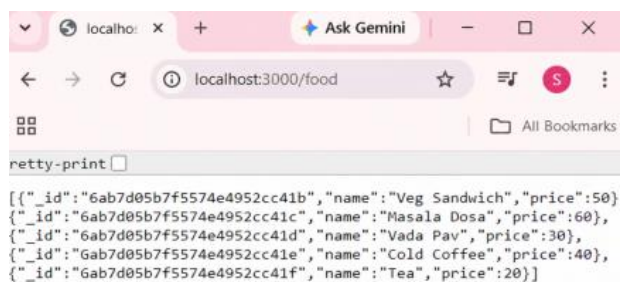
```

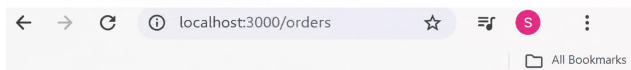
found 0 vulnerabilities
PS D:\MERN PRACTICAL> node server.js
Server is running on http://localhost:3000
MongoDB Connected Successfully

```



College Canteen Pre-Order System API is Running





prac8

localhost:5173

College Canteen Pre-Order System

Available Food Items

Food Item	Price (₹)
Veg Sandwich	₹50
Masala Dosa	₹60
Vada Pav	₹30
Cold Coffee	₹40
Tea	₹20

Student Order Form

Vinayak Name

Roll Number

Course

Select Food Item

Quantity

Placed Orders

Name	Roll No	Course	Food Item	Quantity	Price	Total
Vinayak	37	BSCIT	Masala Dosa	1	₹60	₹60

```
PS D:\MERN PRACTICAL> node server.js
Server is running on http://localhost:3000
MongoDB Connected Successfully
Food items added successfully
```

MongoDB Compass - localhost:27017/CollegeDB

Connections Edit View Help

Compass

My Queries

Data Modeling

CONNECTIONS (1)

Search connections

localhost:27017

- CanteenDB
 - foods
 - orders
- CollegeDB
 - Students
 - food
 - orders
 - students
- admin
- config
- local

CollegeDB

localhost:27017 > CollegeDB

> Open MongoDB shell + Create collection Refresh

Collection name	Properties	Storage size	Data size	Documents	Avg. document size	Indexes	Total index size
Students	-	20.48 kB	89.00 B	1	89.00 B	1	20.48 kB
food	-	20.48 kB	265.00 B	5	53.00 B	1	20.48 kB
orders	-	20.48 kB	131.00 B	1	131.00 B	1	20.48 kB
students	-	36.86 kB	288.00 B	3	96.00 B	1	36.86 kB